Chapter 2: Tools, Setup, and Your First Database

Rod Trent

Table of Contents

Chapter 2: Tools, Setup, and Your First Database

Download this chapter as a PDF for offline reading:
Chapter-02-Tools-Setup-and-Your-First-Database.pdf

Before you can query code, you need the tools and a CodeQL database. This chapter walks through the practical choices of environment, installing the CodeQL CLI (and the Visual Studio Code extension), and creating your first database. By the end you will have a working setup and a database you can use with the `hello-world.ql` query from Chapter 1.

Environments

You can work with CodeQL in several environments. Choose the one that matches how you plan to use it.

Local machine (recommended for learning). Install the CodeQL CLI and the CodeQL extension for Visual Studio Code on your laptop or desktop. This gives you the fastest feedback loop while writing and refining queries. All examples in this book assume a local setup.

Visual Studio Code only. The official CodeQL extension can automatically manage a copy of the CLI for you. This is the lowest-friction way to get started if you primarily want an IDE experience. You can still drop to the terminal when you need the full CLI.

CI / GitHub Actions. Once you are comfortable, the same CLI is used by the CodeQL Action in GitHub Actions for continuous scanning. Local databases and queries transfer directly to CI. We cover workflow integration in a later chapter.

Container / remote. Some teams run the CLI inside Docker or on a build server. This is useful for reproducible builds or when analyzing large monorepos, but it adds complexity you do not need on day one.

For the rest of this book we will use a local CLI installation plus the VS Code extension. If you prefer only the extension, the extension's managed CLI is sufficient for everything that follows.

License note. The CodeQL CLI is free to use on open-source repositories (and for academic research). Commercial or private-code use requires a GitHub Advanced Security / Code Security license. See the GitHub CodeQL Terms and Conditions.

Installing the CLI

The recommended way to install CodeQL is to download the **CodeQL bundle**. The bundle contains the CLI, a compatible set of standard libraries and queries, and precompiled query plans. Using the bundle avoids version mismatches and gives better performance than installing the CLI alone and checking out the `github/codeql` repository separately.

1. Download the bundle

Go to the CodeQL Action releases page and download the per-platform bundle for your machine:

Platform	File
Linux x64	codeql-bundle-linux64.tar.zst
Linux ARM64	codeql-bundle-linux-arm64.tar.zst
macOS	codeql-bundle-osx64.tar.zst
Windows	codeql-bundle-win64.tar.zst

`.tar.gz` variants exist for systems that lack Zstandard support. Prefer the `.zst` files when you can.

2. Extract and add to PATH

Extract the archive to a permanent location (for example `~/codeql` or `C:\codeql`). The executable lives in the `codeql` subdirectory.

Linux / macOS:

```
mkdir -p ~/codeql
tar -xf codeql-bundle-linux64.tar.zst -C ~/codeql
echo 'export PATH="$HOME/codeql/codeql:$PATH"' >> ~/.bashrc    # or ~/.zshrc
source ~/.bashrc
```

Windows (PowerShell):

```
# After extracting to C:\codeql
$env:Path += ";C:\codeql\codeql"
# Persist for future sessions:
[Environment]::SetEnvironmentVariable("Path", $env:Path + ";C:\codeql\codeql",
"User")
```

3. Verify the installation

```
codeql version
codeql resolve languages
codeql resolve packs
```

`resolve languages` lists every language the CLI can create databases for. `resolve packs` confirms that the standard query and library packs shipped with the bundle are visible. If packs are missing, you downloaded the standalone CLI instead of the bundle—re-download the bundle.

A small helper script that runs these checks is available in the companion repository: `examples/chapter02/verify-cli.sh`.

Optional: Visual Studio Code extension

Install the **CodeQL** extension from the Visual Studio Code Marketplace (publisher: GitHub). By default the extension downloads and manages its own CLI. You can also point it at the CLI you just installed so both the IDE and the terminal share the same binary.

Once the extension is installed, open the Command Palette and run **CodeQL: Check for CLI Updates** if you want the latest managed version.

Creating a Database

A CodeQL database is the relational snapshot of a codebase that queries run against. Creating one is the step that turns source code into queryable data.

Interpreted languages (JavaScript, Python, Ruby)

For these languages the extractor reads source files directly. No build step is required.

The companion repository contains a tiny JavaScript project at `examples/chapter02/sample-js/`. From the repository root:

```
codeql database create my-first-db \
  --language=javascript \
  --source-root=examples/chapter02/sample-js
```

(You can also use `--language=javascript-typescript`.)

The command creates the `my-first-db` directory. It must not already exist. Inside you will find:

- A language-specific database directory (e.g. `db-javascript`)
- A source archive that preserves the original files for result highlighting
- Metadata describing the extraction

Confirm success:

```
codeql database info my-first-db
```

Compiled languages (C/C++, Java, Go, C#, Rust, Swift, ...)

These require a successful build so the extractor can observe compilation. Two common patterns:

Autobuild (CodeQL tries to detect the build system):

```
codeql database create my-java-db \
  --language=java \
  --source-root=/path/to/project
```

Manual build command:

```
codeql database create my-cpp-db \
  --language=cpp \
  --command="make -j8" \
  --source-root=/path/to/project
```

Always ensure dependencies are installed and the project builds cleanly *before* you ask CodeQL to trace the build. If the build fails, the database will be incomplete or empty.

Multi-language projects

Use `--db-cluster` when a repository contains several languages:

```
codeql database create my-cluster \
  --db-cluster \
  --language=javascript,python \
  --source-root=.
```

Each language receives its own subdirectory.

Running a query against your new database

Point the Chapter 1 hello-world query at the database you just created:

```
codeql query run \
  --database=my-first-db \
  examples/chapter01/hello-world.ql
```

You should see a single result row containing the string `hello world`.

In VS Code, open the database with the CodeQL extension (**CodeQL: Choose Database**), then open any `.ql` file and run it. Results appear in the CodeQL results view with source highlighting.

Downloading pre-built databases

For many public GitHub repositories you can download a ready-made CodeQL database instead of creating one yourself. From a repository page, look for the Code scanning / CodeQL databases section, or use the GitHub REST API / `gh` CLI. Downloaded databases are ideal when you want to practice queries on large, real-world codebases without waiting for extraction.

Common Issues and Next Steps

Symptom	Likely cause	Fix
codeql: command not found	PATH not set	Re-export PATH or restart the shell
No packs found	Downloaded standalone CLI	Re-download

Symptom	Likely cause	Fix
		the CodeQL **bundle**
Empty or failed database (compiled language)	Build failed or dependencies missing	Build the project manually first, then re-run with --command
macOS Apple Silicon issues	Missing Rosetta or Xcode CLI tools	Install both

You now have a working CodeQL installation and your first database. In the next chapter we look more closely at what is inside a database and how extraction differs between interpreted and compiled languages.

All of the commands and the sample project referenced above live in the companion repository: github.com/rod-trent/MustLearnCodeQL under `examples/chapter02/`.

Must Learn CodeQL — Rod Trent
Companion repository: github.com/rod-trent/MustLearnCodeQL